DGML: the Domain Native Document Format for Agents

Version: 0.71

Specification prepared by:

Name	Company
Jean Paoli	Docugami
Claudio Caldato	Docugami
Andrew Begun	Docugami
Gregory Senay	Docugami
Albert Berdellans	Inveniam
Rick Bunker	Inveniam

---

1. Overview

DGML (Document Graph Markup Language) is a semantic XML representation of business documents. Where raw source files give you layout and pixels, DGML gives you meaning: tags that describe what each element *is* in the document's domain — a contract clause, an invoice line item, a policy definition — not how it appeared on the page.

The headline feature is **cross-document tag consistency**: documents of the same kind share the same semantic vocabulary — what separates DGML from a raw extraction or structural transcription, and what makes it suitable for reasoning over a corpus rather than a single file.

The second property is **complete semantic preservation**. Traditional extraction pipelines choose fields upfront and discard the rest — a decision that fails the moment a new use case emerges and needs a field no one thought to extract. DGML preserves the full semantic structure instead — every element, relationship, and typed value — so a document processed once stays fully queryable without returning to the source.

The third is **document order with graph semantics**. Most graph formats treat documents as unordered collections of facts, but in business documents order is meaning: definitions precede usage, clause sequence governs interpretation, provenance depends on position. DGML

preserves document order as a first-class property while also representing relationships across elements and documents as a graph.

The fourth is **attestation**: **Proof of Origin at the data-element level**. Every DGMLX package is tamper-evident — any alteration to its content breaks its cryptographic hash. The deeper innovation is that this hashing isn't limited to the whole document: because the semantic tree is structured, any XML element subtree — a single data point, a payment term, a liability cap — can be hashed and anchored on an external chain independently, proving its origin without producing the entire document.

Why Existing Formats Fall Short in the Token Economy

The obvious alternatives each have a fundamental problem when applied to enterprise document AI:

Raw text / Markdown forces the agent to re-discover structure on every call. It reads to find connections rather than traversing them, and cannot natively express the relationships inside a long business document without becoming unreadable. Structure that was implicit in the original — the containment of a liability cap inside an indemnification clause, the linkage of a payment term to an effective date — has to be reconstructed by the model from scratch, every time.

Raw PDF or image input burns the full document into every call with no structural leverage — the model must re-read the whole contract to answer a question about one clause.

Structural markup formats are a step up: they replace visual layout with generic structural tags — sections, headings, lists, tables — much like XHTML. This gives agents a navigable tree instead of a flat stream, which helps. But the tags describe *shape*, not *meaning*. A `<section>` containing a liability cap and a `<section>` containing a payment schedule are structurally identical; the agent still has to read and interpret the content to understand what it is looking at. There is no stable vocabulary to query across documents, and no typed values — every amount, date, or identifier remains buried in text.

A Four-Layer Architecture

What DGML gives agents instead is a four-layer architecture that no flat format can replicate:

Semantic layer. Consistent tags across similar documents let agents reason over a corpus without per-document prompt engineering. An agent querying one contract's liability clause can query the same element across a thousand contracts using identical logic. Each tag groups

many surface tokens into a single named concept and any XML element subtree can be sent in isolation to focus the model's attention precisely.

```
<docset:LiabilityCap xsi:type="decimal" dg:value="500000">$500,000</docset:LiabilityCap>
<docset:EffectiveDate xsi:type="date"    dg:value="2024-01-01">January 1, 2024</docset:EffectiveD
<docset:VendorName>MagicSoft, Inc.</docset:VendorName>
```

Spatial layer. Every element carries bounding-box coordinates back to the source page image. Extracted values are not floating assertions — they can be shown to a human as a highlighted region on the original page. This is the difference between an answer and a verifiable answer. Strip the `dg:origin` attributes before sending to a model; keep them in the file for grounding. This single step cuts token count by a further 20–40% on typical documents.

Attestation layer. The DGMLX package (ZIP) contains a `META-INF/dgml-attestation.xml` file that holds a cryptographic hash of the DGML document and its source page images. This makes the package tamper-evident: any alteration to the content breaks the hash.

The deeper innovation is fragment-level attestation. The traditional blockchain use case is hashing an entire document and anchoring that hash on-chain as proof of origin for the whole file. DGML goes further: because the semantic tree is structured, any XML element subtree — a single data element, a payment term, a liability cap — can be hashed independently and anchored on an external chain, establishing Proof of Origin for that specific data artifact. This means a recipient can prove that a specific data element in their copy is identical to the element in the signed original, without needing to produce the entire document. Provenance is at the data-element level, not just the file level.

Readable layer. A DGML file is human-readable in any browser — no special viewer, no extra tooling. Elements can carry `dg:structure` to map to HTML tags and `dg:style` to express observed simple formatting as CSS. A few lines of JavaScript or Python is all it takes to render a DGML document as a styled web page, with no knowledge of the docset schema required.

Additional Capabilities

Composable fragments across organizations. XML namespaces are globally unique and decentralized. Any organization defines its own docset namespace (`http://www.dgml.io/{org}/{docset-name}#`), and DGML fragments from different organizations can be combined into a single document without name collisions. Provenance chains make these cross-organizational assemblies verifiable end-to-end.

A tree and a graph. The XML hierarchy captures containment — sections contain clauses, clauses contain values. Named links across elements capture relationships — an amendment points to what it modifies, a receipt points to the obligation it fulfills. Together they form a JSON-LD-compatible graph derived directly from the XML, with no additional infrastructure.

This combination — semantic tagging, spatial grounding, cryptographic attestation, and human readability — is what makes DGML suitable for enterprise document AI, where the cost of a hallucinated or unverifiable answer is not an acceptable trade-off. DGML is designed for both human operators and LLM agents: structured output, no interactive prompts.

DGMLX (`.dgmlx`) is the packaging format: a self-contained ZIP archive that bundles the source document, its rendered page images, the DGML XML, a formal schema, and attestation metadata. DGMLX is described in Part II.

Design Principles

- **Semantic, not visual.** Tags describe domain meaning; layout is informational only.
- **Consistent across documents.** Tag vocabularies are stable within a docset — the same field name means the same thing in every document.
- **Self-contained.** Everything needed to understand, validate, and verify the document is inside the package.
- **Standard formats throughout.** XML, OPC, RELAX NG, JSON-LD — no proprietary formats.
- **Apache 2.0.** Explicit patent grant for all contributors; no copyleft dependencies in the toolchain.

DGML at a Glance

> A one-page technical summary. The sections that follow specify each concept in full.

What it is. DGML is a semantic XML format for business documents. Tags describe what elements *are* in the document's domain — `<docset:LiabilityCap>`, `<docset:EffectiveDate>` — not how they looked on the page. The full document structure is preserved; nothing is discarded.

Two namespaces at the core.

Prefix	URI	Purpose
`dg:`	`http://dgml.io/ns/dg#`	Core structural elements and attributes
`docset:`	`http://www.dgml.io/{org}/{docset-name}#`	Semantic vocabulary for a document type

Any organization can define its own `docset:` namespace. Multiple namespaces can coexist in one document without collision.

Core elements.

Element	Role
`dg:chunk`	Universal structural primitive — used for all scaffolding that has no domain concept
`dg:extraction`	User-directed structured projection; always a direct child of the root `dg:chunk`. See §13.

Every other element is a `docset:` semantic tag. A wrong semantic tag is worse than an honest `dg:chunk`.

Key attributes.

Attribute	What it carries
`dg:structure`	Visual/layout role: `section`, `header`, `p`, `ol`, `ul`, `li`, `table`, `tr`, `td`, `lim`, `span`
`dg:style`	Inline CSS declarations observed in the source (semicolon-separated; verbatim into HTML `style`)
`dg:origin`	Pixel bounding box(es) of the element's own printed text: `"<page> <x1> <y1> <x2> <y2>"` — traces every element back to its source page image. Boxes only; no reserved values. See §7.
`dg:value`	Normalized machine-readable value (accompanies `xsi:type`)
`dg:itemprop`	Named relationship predicate (JSON-LD-compatible)

Attribute	What it carries
`dg:href`	Relationship target: `#id` (same file), `fileid#id` (cross-file), or semicolon-separated list of targets. Semantic links only. See §8.
`dg:inputs`	The elements a `dg:value` used: `#id` or `fileid#id`, semicolon-separated. See §13.
`dg:method`	How a value the page doesn't print was produced: `calculated`, `inferred`, or `defaulted`. See §13.

Four-layer architecture.

Layer	Mechanism
Semantic	`docset:` tags; consistent vocabulary within a docset; token-efficient for LLMs
Spatial	`dg:origin` on every element; strip before sending to model, keep for verification
Attestation	ZIP (`META-INF/dgml-attestation.xml`): Merkle hash of XML + page images; optional on-chain staking at file or element level for Proof of Origin (§10)
Readable	`dg:structure` maps elements to HTML tags; `dg:style` carries observed simple formatting as CSS. A few lines of JavaScript or Python renders any DGML document in a browser — no schema knowledge required.

Docsets. A docset is a named collection of same-type documents sharing one schema. Namespace: `http://www.dgml.io/{org}/{docset-name}#`. Schema defined in RELAX NG Compact Syntax (`schema.rnc`). Tag names mean the same thing in every document in the docset.

Packaging (DGMLX). `.dgmlx` is a ZIP (OPC convention) containing: source document, `dgml.xml`, `page_images/page_N.png`, `schema.rnc`, `META-INF/dgml-attestation.xml`, `[Content_Types].xml`, `_rels/.rels`.

LLM workflow (summary). (1) Classify into existing docset or create a new one. (2) Read a document sample; identify key concepts. (3) Write `schema.rnc`. (4) Tag documents, applying `dg:origin`, `xsi:type` / `dg:value`, `dg:itemprop` / `dg:href`, and — for values the page doesn't print in full — `dg:inputs` / `dg:method`.

Part I — DGML

2. What DGML Is

A DGML document is a well-formed XML file. Every element belongs to a namespace that identifies who defined it and what it means. The two foundational namespaces are:

- `dg:` — core structural elements and cross-document infrastructure attributes defined by this specification
- `docset:` — semantic vocabulary defined by an organization for a specific document type

These are the baseline, not the limit. Because XML namespaces are globally unique and decentralized, any organization can define its own docset namespace and any DGML document can mix elements from multiple namespaces without collision — enabling fragments from different organizations, different docsets, or different time periods to be composed into a single document. Blockchain-linked provenance chains (Section 11) make such assemblies verifiable end-to-end.

DGML carries no visual or layout semantics. There are no `<bold>`, `<indent>`, or `<column>` elements. A `<docset:PaymentTerms>` describes what the text *is* in the domain of the document, not how it happened to be rendered.

3. Docsets

A **docset** is a named collection of documents of the same type produced by a specific organization — for example, all Master Services Agreements issued by Acme Corp, or all invoices from a supplier. The docset defines the tag vocabulary: which element names exist, what they mean, and how they relate to each other.

Any organization can define its own docsets. The docset namespace URI encodes both the organization and the docset name:

```
http://www.dgml.io/{org}/{docset-name}#
```

Examples:

Organization	Docset name	Namespace URI
Acme Corp	master-services-agreements	`http://www.dgml.io/acme-corp/master-services-agreements#`
City of Springfield	purchase-orders	`http://www.dgml.io/city-of-springfield/purchase-orders#`
MedSupply Inc.	patient-intake-forms	`http://www.dgml.io/medsupply/patient-intake-forms#`

This namespace appears as the `docset:` prefix binding in every DGML file belonging to that docset, and as the `namespace docset = …` declaration in the accompanying `schema.rnc`. Within one docset the namespace URI is fixed — do not create per-document namespaces.

Deciding How to Group Documents

Group into the **same docset** when documents serve the same functional purpose, share 70–80% of their semantic fields, and those fields mean the same thing across every document in the collection.

Split into **separate docsets** when two document types have different field semantics despite similar surface structure, when one organization's documents differ systematically from another's, or when a document type has changed enough over time that old and new versions cannot be compared field-by-field (in which case version the docset name, e.g., `master-services-agreements-v2`).

4. Namespaces

Prefix	URI	Purpose
`dg:`	`http://dgml.io/ns/dg#`	Core elements (`dg:chunk`, `dg:lim`) and attributes (`dg:origin`, `dg:value`, `dg:itemprop`, `dg:href`)
`docset:`	`http://www.dgml.io/{org}/{docset-name}#`	Semantic element vocabulary defined by an organization for a docset
`xhtml:`	`http://www.w3.org/1999/xhtml`	Inline XHTML content (when present)

Prefix	URI	Purpose
`xsi:`	`http://www.w3.org/2001/XMLSchema-instance`	`xsi:type` for typed values

The `docset:` prefix is a convention used throughout this specification as a placeholder. The actual namespace URI varies per organization and docset.

Concept-driven naming

Only elements that carry a domain concept (a section's topic, a named paragraph or row, an inline value) receive a semantic tag. All other structural scaffolding — headings, lists, list items, table cells, enumerators, untyped spans — is emitted as the generic `dg:chunk` element with its layout role in the `dg:structure` attribute.

This is the defining rule of DGML element naming:

Element has a concept	Tag used
Yes	`docset:ConceptName` or `dg:ConceptName` (PascalCase)
No	`dg:chunk`

A structural element (concept or generic) carries exactly one `dg:structure` value. A concept-bearing inline entity carries no `dg:structure` and sits inside a paragraph, header, or list item.

5. Document Structure

`dg:chunk` — Root and Anonymous Container

`<dg:chunk>` serves two roles:

1. **Document root** — the outermost element of every DGML file.
2. **Anonymous structural container** — used at any depth when a grouping of content has a clear structural role but no domain-specific semantic name applies.

Using `dg:chunk` at an inner level signals "this boundary is real, the name is not." A wrong tag is worse than an honest `dg:chunk`. Inside `dg:extraction`, the same reading lets an anonymous `dg:chunk` carry printed text a value used that no schema field extracts (§13).

The root element of a DGML document is `<dg:chunk>` — a generic structural element, not a docset element. All document content is nested inside it. The root carries the namespace declarations.

```
<?xml version='1.0' encoding='utf-8'?>
<dg:chunk xmlns:dg="http://www.dgml.io/ns/dg#"
          xmlns:docset="http://www.dgml.io/{org}/{docset-name}#"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <!-- document content -->
</dg:chunk>
```

Like every DGML element, `dg:chunk` may carry a `dg:structure` attribute:

```
<?xml version='1.0' encoding='utf-8'?>
<dg:chunk xmlns:dg="http://dgml.io/ns/dg#"
          xmlns:docset="http://www.dgml.io/acme-corp/master-services-agreements#"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <docset:IndemnificationClause>
    <dg:chunk dg:structure="header" dg:origin="3 460 139 2090 205">EXHIBIT A</dg:chunk>
    <docset:LiabilityCap xsi:type="decimal" dg:value="500000.00" dg:origin="3 460 220 2090 265">
  </docset:IndemnificationClause>
</dg:chunk>
```

Element Hierarchy

Elements nest to reflect the document's logical structure. A fragment from a Master Services Agreement:

```
<dg:chunk xmlns:dg="http://dgml.io/ns/dg#"
          xmlns:docset="http://www.dgml.io/acme-corp/master-services-agreements#"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <docset:IndemnificationClause>
    <docset:IndemnifyingParty dg:origin="2 460 310 1800 355">Vendor</docset:IndemnifyingParty>
    <docset:IndemnifiedParty  dg:origin="2 460 360 1800 405">Client</docset:IndemnifiedParty>
    <docset:LiabilityCap xsi:type="decimal" dg:value="500000"
                         dg:origin="2 460 410 1800 455">$500,000</docset:LiabilityCap>
    <docset:EffectiveDate xsi:type="date" dg:value="2024-01-01"
                          dg:origin="2 998 710 1466 755">January 1, 2024</docset:EffectiveDate>
  </docset:IndemnificationClause>
  <docset:PaymentTerms>
    <docset:InvoiceCycle dg:origin="4 460 139 1800 184">Net 30</docset:InvoiceCycle>
    <docset:LatePaymentPenalty xsi:type="decimal" dg:value="0.015"
```

```
                              dg:origin="4 460 190 1800 235">1.5% per month</docset:LatePaymentI
  </docset:PaymentTerms>
</dg:chunk>
```

The `dg:structure` Attribute

Any element may carry a `dg:structure` attribute recording its visual role. A generic renderer maps each value to its HTML equivalent with no knowledge of the docset schema. The approved value set is:

`dg:structure` value	Rendered as	Notes
`section`	`<section>`	Semantic section; heading concept is hoisted to the section element name (see below).
`header`	`<header>`	Heading or title line. Always `<dg:chunk dg:structure="header">` — never carries a semantic tag directly.
`p`	`<p>`	Body paragraph.
`ol`	`<ol>`	Ordered list.
`ul`	`<ul>`	Unordered list.
`li`	`<li>`	List item.
`table`	`<table>`	Tabular container.
`tr`	`<tr>`	Table row.
`td`	`<td>`	Data cell (used for both header and data cells).
`lim`	inline	Verbatim enumerator or item marker (e.g. `1.`, `ITEM 9`). Always `<dg:chunk>`, always first child of its parent.
`span`	inline	Inline run with no resolved concept name. Always `<dg:chunk>`. Never dropped.

No other values are valid. In particular: `h1`–`h6` are not used (heading depth is expressed by tree nesting, not numbered values); `th`, `thead`, `tbody`, `blockquote`, `pre`, `div`, `form`, and `figure` do

not appear in DGML output.

Together, `dg:structure` and `dg:style` are what enable the Readable layer (§1): a few lines of JavaScript or Python map each element to its HTML tag and copy observed formatting into the `style` attribute, rendering any DGML document in a browser with no schema knowledge required. Combined with `dg:origin`, a viewer can also overlay each element's bounding box on the original page image.

The `dg:style` Attribute

`dg:style` records visual styling that is visually evident in the source document. Its value is a semicolon-separated list of standard CSS declarations and can be copied verbatim into an HTML `style` attribute with no transformation.

Omit the attribute entirely when no styling is visually evident. Sparse is correct — the attribute is for observable source formatting, not for decorating output.

Allowed properties and values (nothing outside this list):

Property	Allowed values
`font-weight`	`bold` | `normal`
`font-style`	`italic` | `normal`
`font-size`	`0.75em` | `1em` | `1.25em` | `1.5em` | `2em`
`text-decoration`	`underline` | `line-through` | `none`
`text-align`	`left` | `center` | `right` | `justify`
`text-transform`	`uppercase` | `capitalize` | `none`
`color`	Any CSS named color (e.g. `red`, `gray`, `blue`) — no hex values
`background-color`	Any CSS named color — no hex values
`white-space`	`pre` | `normal`

```
<docset:CompanyName dg:structure="span" dg:style="font-weight: bold; color: gray">
  Acme Corp
</docset:CompanyName>
```

```
<docset:TotalAmount dg:structure="td"
    dg:style="font-weight: bold; text-align: right; background-color: lightyellow">
  $42,000
</docset:TotalAmount>

<dg:chunk dg:structure="header" dg:style="font-size: 1.5em; text-transform: uppercase">
  TERMS AND CONDITIONS
</dg:chunk>
```

6. Typed Values

Elements that carry a canonical machine-readable value alongside their display text use `xsi:type` and `dg:value`:

```
<docset:EffectiveDate
    xsi:type="date"
    dg:value="2021-06-15"
    dg:origin="2 1234 890 1890 935">June 15, 2021</docset:EffectiveDate>

<docset:InvoiceTotal
    xsi:type="decimal"
    dg:value="12450.00"
    dg:origin="8 1400 2900 2200 2945">$12,450.00</docset:InvoiceTotal>
```

- `dg:value` holds the canonical machine-readable form.
- The element text content holds the display form as it appears in the source.
- `xsi:type` uses standard XSD type names: `date`, `integer`, `decimal`, `boolean`, plus any standard XSD type.

7. The `dg:origin` Attribute

`dg:origin` records where in the page images an element's own printed text can be found — boxes only. It may appear on any element — leaf or non-leaf. On a leaf it pins a single value; on a parent it spans the full visual extent of the XML element subtree, enabling viewers to highlight an entire clause, table, or section as well as individual fields.

Format

```
dg:origin="<page> <x1> <y1> <x2> <y2>"
```

Multiple boxes for text spanning lines or pages, separated by semicolons:

```
dg:origin="<page> <x1> <y1> <x2> <y2>; <page> <x1> <y1> <x2> <y2>"
```

Field	Type	Meaning
`page`	integer	1-based page number matching `page_images/page_N.png`
`x1`	integer	Left edge of bounding box, pixels
`y1`	integer	Top edge of bounding box, pixels
`x2`	integer	Right edge of bounding box, pixels
`y2`	integer	Bottom edge of bounding box, pixels

Coordinate System

```
(0,0) ──────────────────► x (increases right)
  │
  ▼
  y (increases down)
```

Origin is the top-left corner of the page image, in pixels — standard raster/screen coordinates, opposite to PDF user space.

Pixel dimensions follow from the DPI declared in the package attestation:

```
pixel_width  = round(page_width_inches  × page-image-dpi)
pixel_height = round(page_height_inches × page-image-dpi)
```

Examples

Single value:

```
<docset:VendorAddress
    dg:origin="1 222 2087 615 2120">1648 NW Market St Suite 500</docset:VendorAddress>
```

Multi-line span:

```
<docset:ConfidentialityObligation
    dg:origin="3 209 710 2341 755; 3 209 760 2341 805">
  Each party shall hold the other's confidential information in strict confidence.
</docset:ConfidentialityObligation>
```

Cross-page span:

```
<docset:WarrantyDisclaimer
    dg:origin="4 209 2800 2341 3300; 5 209 50 2341 300">
  THE SOFTWARE IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND...
</docset:WarrantyDisclaimer>
```

Parent spanning a whole clause with children that have their own origins:

```
<docset:IndemnificationClause dg:origin="3 180 400 2360 1200">
  <docset:IndemnifyingParty dg:origin="3 180 450 900 490">Vendor</docset:IndemnifyingParty>
  <docset:LiabilityCap xsi:type="decimal" dg:value="500000"
      dg:origin="3 900 450 1800 490">$500,000</docset:LiabilityCap>
</docset:IndemnificationClause>
```

Using `dg:origin` with LLMs

`dg:origin` attributes are for grounding and verification — not for reasoning. Strip them before sending DGML to a language model:

```
import re
xml_for_llm = re.sub(r'\s+dg:origin="[^"]*"', '', dgml_xml)
```

The original file (with `dg:origin`) is kept for verification; the stripped version is sent to the model. This alone reduces token count by 20–40% on typical documents.

Values the page doesn't print in full

`dg:origin` holds nothing but boxes: valid values are always numeric, and there are no reserved values. A value the page prints only in part keeps the boxes of the printed part; a value the page doesn't print at all has no `dg:origin`. Either way, what the value was produced from is recorded by `dg:inputs`, and how by `dg:method` — see §13. (DGML 0.7's reserved value `"computed"` is retired; Appendix C covers reading 0.7 files.)

A **born-digital** document — one generated from system state, with no page images — has no boxes to record: its values simply omit `dg:origin`, and `dg:inputs` / `dg:method` work unchanged (§13).

Rules

- Elements for which no source match exists (synthesised labels, empty containers) omit `dg:origin`.
- Bounding boxes are derived from word-token bounding boxes, not character-level boxes.
- Coordinates are always in page-image pixel space, never in source document coordinate space.

8. Semantic Links — `dg:itemprop` and `dg:href`

The XML hierarchy captures containment. Named links across elements capture relationships — an amendment points to what it modifies, a payment points to the obligation it fulfills. Together, hierarchy plus links form a JSON-LD-compatible graph derived mechanically from the XML.

Any element can express a named relationship to another element:

Attribute	Borrowed from	Meaning
`dg:itemprop`	XHTML Microdata	Names the property (predicate) this element expresses
`dg:href`	XHTML / JSON-LD	Points to the object — another element identified by `xml:id`

Together they form a subject–predicate–object triple:

```
subject   = the element carrying the attributes (its xml:id)
predicate = dg:itemprop value
object    = dg:href value
```

Reference Syntax

```
dg:href="#id"              ← element with xml:id="id" in the same file
dg:href="fileid#id"        ← element in the file whose attestation file-id is "fileid"
dg:href="#id1; #id2"       ← semicolon-separated list of targets
```

Cross-file references are stable regardless of filename or path. The semicolon-separated form is valid wherever an element relates to more than one target. Writers join references with `"; "`; readers split a reference list on semicolons or whitespace. The same syntax serves `dg:inputs` (§13).

`dg:itemprop` / `dg:href` express **semantic links only**: a relationship between two elements in the document's domain. They never record provenance — what a value was produced from is `dg:inputs` (§13) — so an element may carry both a link and `dg:inputs`.

Example

```
<docset:VendorName xml:id="vendor-co"
    dg:origin="1 220 150 680 200">MagicSoft, Inc.</docset:VendorName>

<docset:SignatoryName
    xml:id="sig-1"
    dg:itemprop="signatoryOf"
    dg:href="#vendor-co"
    dg:origin="11 460 2430 1200 2475">Jane Smith</docset:SignatoryName>
```

JSON-LD Translation

See Appendix A for the complete JSON-LD export specification, including how `dg:itemprop` / `dg:href` stubs are translated to named link properties.

Cross-file references resolve using the target `file-id` as the URI base. A collection of DGML files in a docset forms a traversable JSON-LD graph. The XML is the ground truth; JSON-LD is derived from it mechanically.

Common link patterns

Amendment — an element in an amendment file references the clause it modifies:

```
<!-- original contract -->
<docset:NoticePeriod xml:id="notice-1" dg:origin="6 460 800 1800 840">30 days</docset:NoticePeri

<!-- amendment file -->
<docset:NoticePeriod
    dg:itemprop="amends" dg:href="5kqt9r5fowno#notice-1"
    dg:origin="2 460 310 1800 350">45 days</docset:NoticePeriod>
```

Fulfillment — links an obligation to its completion event in a later document:

```
<docset:DeliveryObligation xml:id="del-1" dg:origin="4 460 500 1800 540">
  Vendor shall deliver by March 31.
</docset:DeliveryObligation>

<docset:DeliveryReceipt
    dg:itemprop="fulfills" dg:href="5kqt9r5fowno#del-1"
    dg:origin="1 460 200 1800 240">Received March 28.</docset:DeliveryReceipt>
```

Incorporation by reference — a clause in one document incorporates content from another:

```
<docset:GovernanceLaw
    dg:itemprop="incorporates" dg:href="nda-file-id#confidentiality-1"
    dg:origin="7 460 600 1800 640">See Master Agreement §12.</docset:GovernanceLaw>
```

9. Reserved `dg:` Attributes

Attribute	Applies to	Meaning
`dg:origin`	Any element	Pixel bounding box(es) of the element's printed text (Section 7)
`dg:value`	Any element	Canonical machine-readable value
`dg:structure`	Any element	Visual/layout role (Section 5); see approved value set
`dg:style`	Any element	Inline CSS styling observed in the source (Section 5)

Attribute	Applies to	Meaning
`dg:itemprop`	Any element	Property name for a semantic link (Section 8)
`dg:href`	Any element	Target(s) of a semantic link; `#id` or `fileid#id`
`dg:inputs`	Elements with `dg:value`	The elements the value used; `#id` or `fileid#id` list (Section 13)
`dg:method`	Elements with `dg:value` and no `dg:origin`	`calculated`, `inferred`, or `defaulted` (Section 13)

Examples:

```
<docset:CompanyName dg:structure="span" dg:style="font-weight: bold; color: gray">
  Acme Corp
</docset:CompanyName>

<docset:TotalAmount dg:structure="td"
    dg:style="font-weight: bold; text-align: right; background-color: lightyellow">
  $42,000
</docset:TotalAmount>

<dg:chunk dg:structure="header" dg:style="font-size: 1.5em; text-transform: uppercase">
  TERMS AND CONDITIONS
</dg:chunk>
```

10. Blockchain Anchoring

DGML supports on-chain notarization at two levels of granularity: **file-level** (committing to the entire document) and **element-level** (committing to a single XML element). Only hashes, a URI, and a small JSON blob go on-chain — document content never does.

How anchoring works

The anchor-precompile registry contract stores records. Every record is the same 10-field shape regardless of whether it covers a whole file or a single element:

Field	Supplied by	Description
`registry`	caller	Named registry on the chain
`uri`	caller	`dgmlx://` locator (see below)
`checksum`	caller	64-char lowercase hex SHA-256
`checksum_algo`	caller	Always `"sha256"`
`metadata`	caller	Compact JSON (see below)
`timestamp`, `status`, `record_id`, `index`, `is_latest`	chain	Assigned on write

The `META-INF/dgml-attestation.xml` inside the DGMLX bundle is a local Merkle manifest — it is not modified by anchoring. The on-chain record is a separate artifact produced at anchoring time and identified by its URI.

Each URI is versioned: anchoring the same URI again produces a new record rather than overwriting the prior one. All records for a URI remain on-chain. A prior record will no longer verify against a document that has since been regenerated; this is intentional — the chain preserves a complete audit trail of every anchored state.

File-level anchoring

A file stake commits to the entire state of a document: its original bytes, every rendered page image, its docset schema, and its DGML XML. These four artifacts are called **slots**. Each slot is hashed independently, and the four hashes are combined into a single RFC 6962 binary Merkle root (the same construction used by Certificate Transparency logs). That root is what goes on-chain as the `checksum`.

URI: `dgmlx://<file_id>` when the file has not been assigned to a docset, or `dgmlx://<file_id>/<docset_id>` when it has.

Slots: the four slots are always processed in this fixed order:

Slot	What it represents	What is hashed
`source`	The original file as uploaded — the PDF, Word document, or other input.	SHA-256 of the raw file bytes.
`page_image[N]`	The rendered page images — one PNG per page, produced by the workspace at file-add time.	SHA-256 of each page PNG, one leaf per page, ordered by page number (not filename).
`schema`	The RELAX NG docset schema that governs how this file's DGML was generated. It defines the vocabulary of semantic tags used in the XML.	SHA-256 of the raw `.rnc` file bytes.
`dgml`	The DGML XML document itself — the semantic representation of the file's content.	RFC 6962 Merkle root of the XML element tree (the same tree used for element-level staking).

A slot that is absent is omitted from the computation entirely — it contributes no placeholder hash. A file that has not been assigned to a docset has no `schema` or `dgml` slot; only `source` and `page_image[N]` are included. The `slots` field in metadata records how many slots were included so a verifier knows what to expect.

metadata: `{"kind": "dgmlx", "slots": <count>}`

Verification: to verify a file stake, re-derive the Merkle root from the current workspace artifacts using the same slot order and compare it to the on-chain `checksum`. No inclusion proof is stored; the verifier recomputes the full root locally from the artifacts it has. Any byte-level change to any artifact — one character in the XML, one pixel in a page image, one field renamed in the schema — changes the root and fails verification.

Element-level anchoring

An element stake commits to a single XML element, with an inclusion proof that ties it to the document root without revealing any other element.

URI: `dgmlx://<file_id>/<docset_id>#<leaf_index>`

The fragment `#<leaf_index>` is the 0-based DFS pre-order position of the element — the same integer as `proof.leaf_index`.

checksum: the element's canonical hash:

```
SHA-256( exclusive_C14N(element) )
```

Exclusive C14N (W3C XML-C14N 1.0, `exclusive=True, with_comments=False`) ensures the hash is independent of inherited namespace context — the same element hashes identically whether staked standalone or inside a full document.

metadata:

```
{
  "kind": "dgml-element",
  "root_hash": "<64-char hex>",
  "proof": {
    "leaf_hash":  "<64-char hex>",
    "leaf_index": 3,
    "leaf_count": 47,
    "path": [
      {"sibling": "<64-char hex>", "side": "L"},
      {"sibling": "<64-char hex>", "side": "R"}
    ]
  }
}
```

`root_hash` is the Merkle root of the whole DGML XML element tree — the same value the DGMLX bundle records as the `dgml` slot hash. The `checksum` field on the chain record holds the leaf hash; the root sits in `metadata.root_hash`. This is the inverse of a file stake, where `checksum` is the root.

Proof field definitions

A binary Merkle tree hashes data items at its bottom level. Those bottom-level items are called **leaves** — standard graph-theory vocabulary for terminal Merkle tree nodes, used consistently in RFC 6962 (Certificate Transparency), Bitcoin SPV, and Ethereum light clients. Internal Merkle tree nodes are computed by hashing pairs of children upward until a single **root** remains.

The JSON field names below are defined by this specification. The underlying algorithm — binary Merkle tree construction and audit-path verification — follows RFC 6962 §2.

Field	Type	RFC reference	Description
`leaf_hash`	64-char hex	—	The SHA-256 of `exclusive_C14N(element)` for the staked element. This is the element's cryptographic identity in the tree — it captures exactly what the element contains, independent of its position. It is identical to the `checksum` field on the chain record. A verifier recomputes this from the element's XML; if they differ, the element itself has changed since it was staked.
`leaf_index`	integer	RFC 6962 §2.1	The 0-based position of this element among all leaves, counted in DFS pre-order traversal of the full XML element tree. This is how the element is located within the tree. It is also used as the URI fragment (`#<leaf_index>`), so a URI of the form `dgmlx://<file_id>/<docset_id>#3` uniquely identifies this exact element within this exact file.
`leaf_count`	integer	RFC 6962 §2.1.3	The total number of leaves in the tree at the time the stake was created — the tree's size. A verifier uses this to detect tree-shape forgery: an attacker who knows only a root hash could in principle construct a different tree that produces the same root, but cannot do so if the expected leaf count is also checked and must match.
`path`	array	RFC 6962 §2.1.1	The ordered list of sibling hashes needed to re-derive the Merkle root starting from the leaf — called an *audit path* in RFC 6962. The verifier starts with `leaf_hash` and at each step concatenates it with the sibling hash (left or right, as indicated by `side`), then hashes the result. After all steps, the result must equal `root_hash`. Only O(log N) siblings are needed, so the full tree is never stored or transmitted. Each path entry is `{"sibling": "<64-char hex>", "side": "L"|"R"}`.

Verification: given only the element's XML, the proof path, and the root hash from metadata — no other document content is required:

```
candidate = SHA-256( exclusive_C14N(element) )
if candidate != proof.leaf_hash:
    return False          # the element itself changed
current = candidate
for step in proof.path:
    if step.side == "L":
        current = SHA-256( bytes.fromhex(step.sibling) + bytes.fromhex(current) )
    else:
        current = SHA-256( bytes.fromhex(current) + bytes.fromhex(step.sibling) )
return current == root_hash
```

A failed verification distinguishes two cases. If the re-hashed element differs from `proof.leaf_hash`, the element itself changed. If the element hash matches but the path fails to reconstruct `root_hash`, the tree around it changed.

The two levels compose

A file stake records the `dgml` slot's Merkle root as part of the file checksum. An element stake records a single element's hash and carries that same XML Merkle root in `metadata.root_hash`. The two records are independently verifiable but cryptographically consistent: the element proof re-walks to the same root that the file bundle attests.

11. LLM Guidance — Creating DGML

The first decision is always **docset placement**: which docset does this document belong to, and does that docset already exist? Only after that question is answered does schema and tagging work begin.

Step 1 — Determine the Docset

Two situations arise:

Existing docset. A docset namespace already exists with a defined schema (`schema.rnc`). The task is classification: read the document and decide whether it fits an existing docset. If it does, proceed directly to Step 3 (tagging) using the existing schema — do not modify the schema unless the document introduces a concept genuinely absent from it. Consistent schemas

across documents are the core value of DGML; schema drift destroys cross-document queryability.

New docset. No matching docset exists. Define a new namespace following the `http://www.dgml.io/{org}/{docset-name}#` convention, then proceed to Steps 2–4. The docset name should describe the document type, not the organization (`master-services-agreements`, not `acme-legal-docs`).

To decide whether a document fits an existing docset, check: - Same document type (contract, invoice, lease, …)? - Same issuing organization or one with equivalent vocabulary? - Do the key fields map cleanly onto existing schema tags, with no major structural gaps?

If any answer is no, create a new docset rather than forcing the document into an ill-fitting schema.

Step 2 — Read and Understand the Documents

Before writing any schema, read a representative sample of the docset's documents top-to-bottom as a human would. Identify:

1. **The document type** — what is it legally, commercially, or administratively?
2. **The parties** — who are the named entities and what roles do they play?
3. **The key facts** — dates, amounts, identifiers, obligations, terms.
4. **The natural clusters** — how are facts grouped? What are the logical containment relationships?

Step 3 — Define the Schema (new docsets only)

Write a `schema.rnc` that names every recurring field and assigns it a `parentRole` group. The schema is the contract between documents in the docset — it must be stable enough that every document can be tagged against it without revision.

Tag names must describe **what a value is**, not where it appears or how it looks:

✅ Good — describes meaning	❌ Bad — describes position or style
`docset:EffectiveDate`	`docset:TopRightDate`
`docset:LiabilityCap`	`docset:BoldAmountInSection3`
`docset:GovernanceLawJurisdiction`	`docset:LastLineOfPageTwo`

✅ Good — describes meaning	❌ Bad — describes position or style
`docset:VendorTaxId`	`docset:TableColumn2Row4`

Additional rules:

- **Be specific.** `docset:TerminationNoticePeriod` is better than `docset:NoticePeriod` if the document distinguishes notice types.
- **Be stable.** A tag must mean the same thing across every document in the docset. If a slot sometimes means one thing and sometimes another, split it into two tags.
- **Don't mimic layout.** Nesting should reflect real containment or logical grouping.
- **Use `dg:chunk` when uncertain.** A bounded block with an ambiguous semantic role is better wrapped in `dg:chunk dg:structure="…"` than forced into a wrong tag.

Structure and style conventions

These rules govern how `dg:structure` and `dg:style` attribute values are applied when generating DGML. They are generation rules, not schema rules — the schema defines concept names; these rules define how structural scaffolding and visual presentation are expressed.

When to emit `dg:style`. Emit `dg:style` only when a styling property is visually evident in the source page image — bold text, a shaded cell, an uppercase heading, a centered total. Do not emit it to reflect inferred importance, hierarchy, or default rendering. When in doubt, omit it. Apply it at the most specific element where the style is observable: a bold amount goes on `<docset:TotalAmount>`, not on its enclosing row. Combine properties in a single attribute value rather than emitting multiple elements. Use only the allowed property/value pairs from the table in §9.

Heading concept hoisting. A heading is always `<dg:chunk dg:structure="header">` — it never carries a semantic tag directly. The concept identified for the heading is *hoisted* to name the enclosing section element. A section whose heading concept is `SpecialConditions` becomes `<docset:SpecialConditions dg:structure="section">`; a section whose heading has no resolved concept becomes `<dg:chunk dg:structure="section">`.

```
<docset:SpecialConditions dg:structure="section">
  <dg:chunk dg:structure="header">
    <dg:chunk dg:structure="lim">ITEM 9</dg:chunk>Special Conditions</dg:chunk>
  <!-- section content follows -->
</docset:SpecialConditions>
```

List naming. Two patterns apply depending on item length and uniformity:

Uniform short-item lists — all items share one role and are each short (a value or phrase, not a sentence). The container takes a plural collection concept; every item takes the singular:

```
<docset:Annexures dg:structure="ol">
  <docset:Annexure dg:structure="li"><dg:chunk dg:structure="lim">A.</dg:chunk>Plan of the Premi
  <docset:Annexure dg:structure="li"><dg:chunk dg:structure="lim">B.</dg:chunk>Rules and Regulat
</docset:Annexures>
```

Prose / mixed-length lists — items are sentence- or paragraph-length, or vary in length. Each item carries its own per-item concept (or none) and the container stays `dg:chunk`:

```
<dg:chunk dg:structure="ol">
  <docset:RepairObligation dg:structure="li">
    <dg:chunk dg:structure="lim">5.1</dg:chunk>The Lessee must keep the Premises in good repair.
  <docset:InsuranceObligation dg:structure="li">
    <dg:chunk dg:structure="lim">5.2</dg:chunk>The Lessee must insure the Premises for full repl
</dg:chunk>
```

Enumerators and markers (`lim`). Section numbers, list markers, and other delimiter tokens that belong to the layout but carry no domain meaning are wrapped in `<dg:chunk dg:structure="lim">`. Always the first child of their parent element. Never omitted.

```
<docset:PaymentTerms dg:structure="section">
  <dg:chunk dg:structure="header">
    <dg:chunk dg:structure="lim">4.2</dg:chunk>Payment Terms</dg:chunk>
  <docset:InvoiceCycle dg:origin="4 460 139 1800 184">Net 30</docset:InvoiceCycle>
</docset:PaymentTerms>
```

Tables. Tables are often messy: merged cells, rotated headers, implicit column meanings, footnotes in cells. The goal is to capture the *information* semantically, not reproduce the layout.

Tables fall into two archetypes:

Record table — every row is the same kind of thing (item / price / quantity). The table takes a plural collection concept; each row takes the singular. Column concepts are specific role names (`ProductName`, `UnitPrice`), never bare generics (`Name`, `Amount`) and never structural words (`Item`, `Row`). The "Table" suffix is stripped (`BilledCharges`, not `BilledChargesTable`).

```
<docset:BilledCharges dg:structure="table">
  <docset:BilledCharge dg:structure="tr">
    <docset:ProductName dg:structure="td">Widget</docset:ProductName>
    <docset:UnitPrice dg:structure="td" xsi:type="decimal" dg:value="9.95">$9.95</docset:UnitPri
    <docset:OrderedQuantity dg:structure="td" xsi:type="integer" dg:value="3">3</docset:OrderedQu
  </docset:BilledCharge>
</docset:BilledCharges>
```

Key-value table — each row is a different field (label cell + value cell); column position carries no shared meaning. The label cell stays `dg:chunk`; the value cell takes its own per-row concept.

```
<docset:Premises dg:structure="tr">
  <dg:chunk dg:structure="td">ITEM 1 — Premises</dg:chunk>
  <docset:PremisesDescription dg:structure="td">Portion of the land…</docset:PremisesDescription>
</docset:Premises>
```

The archetype is inferred: if a column's concept repeats across rows → record; if it varies → key-value. In both cases: normalize values with `dg:value`, and trace every cell with `dg:origin`.

Step 4 — Tag the Documents

Apply the schema to each document, producing a DGML XML file. At this stage:

- Assign `dg:origin` to every element where the source location is known.
- Assign `xsi:type` and `dg:value` to every element whose content can be normalized.
- Use `dg:itemprop` / `dg:href` to express relationships between elements.
- For a value the page doesn't print in full, record what it used in `dg:inputs` and, when the page doesn't print it at all, how it was produced in `dg:method` (§13).
- Use `dg:chunk` wherever a structural boundary is real but no schema tag applies.

12. Schema — `schema.rnc`

The docset vocabulary is formally defined in **RELAX NG Compact Syntax** (ISO/IEC 19757-2). This is the artifact generated in Step 2 of the LLM workflow (Section 12) — the schema comes first, before any document is tagged. The namespace declaration matches the `docset:` binding in all DGML files in the docset:

```
namespace docset = "http://www.dgml.io/acme-corp/master-services-agreements#"
```

`parentRole` — encoded as named pattern groups:

```
ContractLevelTag = IndemnificationClause | PaymentTerms | GovernanceLaw | ...
ClauseItemTag    = LiabilityCap | EffectiveDate | VendorName | ...
InlineTag        = VendorTaxId | InvoiceTotal | SignatoryName | ...

AnyTag = ContractLevelTag | ClauseItemTag | InlineTag
```

Element definitions carry `##` doc comments. Three annotations are defined:

Annotation	Purpose
`## <description>`	Human-readable description of the element
`## Example: …`	Representative value
`## Prompt: …`	For values read off the page: guides the LLM where to find the value. For values the page doesn't print in full — printed in part, calculated, inferred, or defaulted — describes the derivation rule. See §13.

```
## Agreed maximum liability exposure under the contract
## Example: $500,000
## Prompt: Look for the dollar amount following "liability" or "not exceed" in the indemnificati
LiabilityCap =
  element docset:LiabilityCap {
    (text | AnyTag)*
  }
```

Translation to JSON-LD

See Appendix A for the schema vocabulary JSON-LD export, including the context definition and concept mapping table.

---

13. User-Directed Schema and Extraction

By default, an LLM tagging a docset invents its own vocabulary. User-directed extraction lets the user seed that vocabulary — naming some or all of the concepts, guiding where to find

them, and optionally specifying which data elements to pull out and how to arrange them — while the LLM fills in the rest.

Overview

User-directed extraction starts with a `schema.rnc` extended with `## Prompt:` annotations (§12). If a matching docset already exists, the LLM uses it directly — no schema writing required. For a new docset, the user provides a schema, either seeding the full vocabulary or targeting specific fields. In both cases the LLM may enrich the schema with additional tags it discovers.

Output can be a full document tree, a targeted `dg:extraction` (§5), or both in the same file. The schema and its `## Prompt:` annotations drive all three forms equally.

Always a Proper Docset

Even for pure targeted extraction, the user defines a namespace URI following `http://www.dgml.io/{org}/{docset-name}#`. There are no shortcuts. The named docset is what guarantees cross-document consistency at scale: an extraction schema used against a thousand invoices produces a thousand `dg:extraction` elements with identical tag names and structure.

The Input Schema

The input schema is a standard `schema.rnc` (§12) extended with `## Prompt:` annotations and explicit content models. For values read off the page, `## Prompt:` guides the LLM where to find the value. For values the page doesn't print in full, it describes the derivation rule — the rule a verifier reapplies (see Value Provenance below).

RNC invoice example:

```
namespace docset = "http://www.dgml.io/acme/invoices#"

## Invoice root
Invoice =
  element docset:Invoice {
    (VendorName | InvoiceDate | LineItems | InvoiceTotal)*
  }

## Legal name of the vendor
## Example: MagicSoft, Inc.
## Prompt: Look for the company name at the top of the invoice
```

```
VendorName =
  element docset:VendorName {
    text
  }

## Invoice issue date
## Example: 2025-03-01
## Prompt: Look for the date labeled "Invoice Date" or "Date"
InvoiceDate =
  element docset:InvoiceDate {
    text
  }

## Collection of line items
LineItems =
  element docset:LineItems {
    LineItem*
  }

## Single line item
LineItem =
  element docset:LineItem {
    (ProductName | Quantity | UnitPrice)*
  }

## Product or service name
## Prompt: The description column of the line item
ProductName =
  element docset:ProductName {
    text
  }

## Quantity ordered
## Prompt: The quantity column of the line item
Quantity =
  element docset:Quantity {
    text
  }

## Unit price
## Prompt: The unit price column of the line item
UnitPrice =
  element docset:UnitPrice {
    text
  }

## Total invoice amount — calculated as the sum of (Quantity × UnitPrice) across all LineItems
```

```
## Prompt: Compute as sum of Quantity × UnitPrice for each LineItem; verify against any printed t
InvoiceTotal =
  element docset:InvoiceTotal {
    text
  }
```

Equivalent JSON-LD context (A.3 form):

```
{
  "@context": {
    "docset": "http://www.dgml.io/acme/invoices#",
    "xsd": "http://www.w3.org/2001/XMLSchema#",
    "Tag":          "docset:Tag",
    "TagGroup":     "docset:TagGroup",
    "members":      { "@id": "docset:members", "@type": "@id", "@container": "@set" },
    "description":  "docset:description",
    "example":      "docset:example",
    "prompt":       "docset:prompt",
    "children":     { "@id": "docset:children", "@type": "@id", "@container": "@list" }
  },
  "@graph": [
    {
      "@id": "docset:Invoice",
      "@type": "docset:Tag",
      "description": "Invoice root",
      "children": ["docset:VendorName", "docset:InvoiceDate", "docset:LineItems", "docset:Invoic
    },
    {
      "@id": "docset:VendorName",
      "@type": "docset:Tag",
      "description": "Legal name of the vendor",
      "example": "MagicSoft, Inc.",
      "prompt": "Look for the company name at the top of the invoice"
    },
    {
      "@id": "docset:InvoiceDate",
      "@type": "docset:Tag",
      "description": "Invoice issue date",
      "example": "2025-03-01",
      "prompt": "Look for the date labeled \"Invoice Date\" or \"Date\""
    },
    {
      "@id": "docset:LineItems",
      "@type": "docset:Tag",
      "description": "Collection of line items",
```

```
      "children": ["docset:LineItem"]
    },
    {
      "@id": "docset:LineItem",
      "@type": "docset:Tag",
      "description": "Single line item",
      "children": ["docset:ProductName", "docset:Quantity", "docset:UnitPrice"]
    },
    {
      "@id": "docset:ProductName",
      "@type": "docset:Tag",
      "description": "Product or service name",
      "prompt": "The description column of the line item"
    },
    {
      "@id": "docset:Quantity",
      "@type": "docset:Tag",
      "description": "Quantity ordered",
      "prompt": "The quantity column of the line item"
    },
    {
      "@id": "docset:UnitPrice",
      "@type": "docset:Tag",
      "description": "Unit price",
      "prompt": "The unit price column of the line item"
    },
    {
      "@id": "docset:InvoiceTotal",
      "@type": "docset:Tag",
      "description": "Total invoice amount",
      "prompt": "Compute as sum of Quantity × UnitPrice for each LineItem; verify against any pr
    }
  ]
}
```

Output Modes

Three output modes are defined. Implementations must use these canonical identifiers.

Mode	Identifier	Output	Nesting
1	`full`	Document tree in root `dg:chunk`	Follows the document structure

Mode	Identifier	Output	Nesting
2	`full-extraction`	Document tree in root `dg:chunk` + `dg:extraction` as sibling	Tree follows document; extraction follows user schema
3	`extraction`	`dg:extraction` inside root `dg:chunk`	Follows user schema

In mode `full-extraction`, values inside `dg:extraction` carry `dg:origin` (pointing to the page) and may also link to the corresponding element in the main document tree with `dg:itemprop` / `dg:href` (§8). In mode `extraction`, `dg:origin` is the only link to the page.

`dg:extraction` Placement and Rules

- Always a **direct child** of the root `dg:chunk` — never nested inside the document tree.
- Never carries its own `xmlns:docset` declaration — inherits from the root `dg:chunk`.
- Content is 100% standard DGML: `docset:` elements, `xsi:type` / `dg:value`, `dg:origin`, `dg:inputs` / `dg:method`, `dg:itemprop` / `dg:href`.
- Nesting inside `dg:extraction` follows the user's schema, not the document's physical structure.
- Anonymous `dg:chunk` elements may appear next to the fields that use them, carrying printed text a value used that no schema field extracts (see Value Provenance below).

Value Provenance — `dg:origin`, `dg:inputs`, `dg:method`

Every extracted value says where it came from, with up to three attributes, each answering one question:

Attribute	Question it answers
`dg:origin` (§7)	Where is the element's own text printed? Boxes only.
`dg:inputs`	Which elements did the value use?
`dg:method`	How was a value the page doesn't print produced? `calculated`, `inferred`, or `defaulted`.

Rules:

1. `dg:origin` holds the boxes of the element's own printed text, nothing else.
2. `dg:inputs` lists the elements that fed `dg:value`, as `#id` or `fileid#id` joined with `"; "` (§8). It appears only on elements with a `dg:value`. Every target must exist; no self-references or cycles.
3. `dg:method` (optional) says how a value the page doesn't print was produced. It is allowed only on elements with a `dg:value` and no `dg:origin`. `calculated` and `inferred` require `dg:inputs`; `defaulted` forbids it:
   - `calculated` — a deterministic calculation over `dg:inputs`, which a verifier can redo.
   - `inferred` — a classification or judgment from `dg:inputs`; the inputs can be checked, the reasoning can't.
   - `defaulted` — a producer default, such as a value the field's `## Prompt:` names for when the document says nothing (a sum over no line items is 0 by default).
4. **When `dg:method` is absent**, readers treat a value with `dg:inputs` as `inferred` — the safe reading — and a value without it as unsupported.
5. **The derivation rule lives in the schema**, in the field's `## Prompt:` annotation (§12). For example: "Printed as month/day only. Take the year from CommencementDate, using the next year when the month is earlier than the commencement month."
6. **`dg:itemprop` / `dg:href` are semantic links only** (§8). They never carry provenance, so an element can have both a link and `dg:inputs`.
7. **A producer emits every element it uses as an input.** When an input is already an element in the file, as in full-tree output, the producer references it, adding an `xml:id` if it has none. When it isn't, as in extraction-only output, the producer emits it as an anonymous `dg:chunk` with an `xml:id`, its `dg:origin`, and its printed text, next to the field that uses it.

A producer should not write a value it read off the page but could not locate: without `dg:origin` it would read as unsupported, or — with `dg:inputs` — as an inferred judgment. Boxed values never need `dg:method`: the boxes and `dg:inputs` already say how they were read.

Reading an element. Two attributes give four cases; `dg:method` refines only the bottom row, where the page prints nothing.

	No `dg:inputs`	**Has `dg:inputs`**
Has `dg:origin`	Read from its printed text ("$4,200.00" → 4200.00)	Printed in part, completed from the inputs ("3/1" + the commencement date)

	No `dg:inputs`	**Has `dg:inputs`**
No `dg:origin`	`dg:method="defaulted"`: a producer default. Without it: unsupported	`dg:method="calculated"` or `"inferred"`. Without it: read as `inferred`

A value printed only in part is one element: its boxes cover the printed part, and `dg:inputs` names what completed it.

```
<docset:PeriodStart xml:id="period-start" xsi:type="date" dg:value="2026-03-01"
    dg:origin="2 410 980 520 1025" dg:inputs="#commencement-date">3/1</docset:PeriodStart>

<docset:PeriodMonths xsi:type="integer" dg:value="12"
    dg:inputs="#period-start; #period-end" dg:method="calculated">12</docset:PeriodMonths>

<docset:LateFeeGraceDays xsi:type="integer" dg:value="0"
    dg:method="defaulted">0</docset:LateFeeGraceDays>
```

A currency printed only as "$" is the same case: `dg:origin` boxes the symbol wherever it appears (several boxes mean the same text printed more than once), and `dg:inputs` names what settles which dollar it is. If no symbol is printed, the currency is `inferred` from the element that decides it — an address, the issuer, a tax line — or `defaulted` when nothing in the document decides it. `dg:inputs` never points at the whole document, because that gives a verifier nothing to check.

```
<docset:Currency dg:value="CAD"
    dg:origin="2 1510 880 1528 915; 2 1510 1240 1528 1275"
    dg:inputs="#premises-address">$</docset:Currency>
```

Inputs that aren't schema fields (rule 7). A schema that extracts only `EndDate` meets a contract that prints a start date and a term, but no end date. `EndDate` is calculated, and its inputs become anonymous chunks — so the schema stays clean, and the field reads the same across documents: boxed where a contract prints it, calculated from chunks where it doesn't.

```
<dg:chunk xml:id="start" dg:origin="1 300 410 620 450">January 15, 2026</dg:chunk>
<dg:chunk xml:id="term" dg:origin="1 300 520 980 560">a term of twenty-four (24) months</dg:chunk

<docset:EndDate xsi:type="date" dg:value="2028-01-14"
    dg:inputs="#start; #term" dg:method="calculated">2028-01-14</docset:EndDate>
```

Chunks are needed only in extraction-only output; with the full document tree, the inputs already exist there.

Born-digital documents. A value in a document generated from system state, with no page images, omits `dg:origin`; `dg:inputs` and `dg:method` work unchanged. The one difference from a paged document is the bottom-left case: with page images, a value with no boxes and no inputs is unsupported; without them, it is source data. A reader therefore needs to know, per file, whether page images exist — in a DGMLX package, the package says so. Verification relies on the attestation hash instead of cropping, plus recomputing `calculated` values.

Verifying an element. A verifier crops the boxes and compares the crop with the element's text (§20), then follows `dg:inputs` and reapplies the schema's `## Prompt:` rule. It recomputes `calculated` values, checks only that the inputs of `inferred` values exist, and flags `defaulted` and unsupported values.

Full Example — Mode 3 (Extraction Only)

```
<?xml version="1.0" encoding="UTF-8"?>
<dg:chunk xmlns:dg="http://dgml.io/ns/dg#"
          xmlns:docset="http://www.dgml.io/acme/invoices#"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

  <dg:extraction>
    <docset:Invoice>
      <docset:VendorName
          dg:origin="1 220 150 680 200">MagicSoft, Inc.</docset:VendorName>
      <docset:InvoiceDate
          xsi:type="date" dg:value="2025-03-01"
          dg:origin="1 900 150 1300 200">March 1, 2025</docset:InvoiceDate>
      <docset:LineItems>
        <docset:LineItem>
          <docset:ProductName
              dg:origin="2 200 400 900 440"
              xml:id="ln1-name">Widget Pro</docset:ProductName>
          <docset:Quantity
              xsi:type="integer" dg:value="3"
              dg:origin="2 900 400 1100 440"
              xml:id="ln1-qty">3</docset:Quantity>
          <docset:UnitPrice
              xsi:type="decimal" dg:value="49.95"
              dg:origin="2 1100 400 1400 440"
              xml:id="ln1-price">$49.95</docset:UnitPrice>
        </docset:LineItem>
```

```
        <docset:LineItem>
          <docset:ProductName
              dg:origin="2 200 450 900 490"
              xml:id="ln2-name">Support Plan</docset:ProductName>
          <docset:Quantity
              xsi:type="integer" dg:value="1"
              dg:origin="2 900 450 1100 490"
              xml:id="ln2-qty">1</docset:Quantity>
          <docset:UnitPrice
              xsi:type="decimal" dg:value="200.00"
              dg:origin="2 1100 450 1400 490"
              xml:id="ln2-price">$200.00</docset:UnitPrice>
        </docset:LineItem>
      </docset:LineItems>
      <docset:InvoiceTotal
          xsi:type="decimal" dg:value="349.85"
          dg:inputs="#ln1-qty; #ln1-price; #ln2-qty; #ln2-price"
          dg:method="calculated">$349.85</docset:InvoiceTotal>
    </docset:Invoice>
  </dg:extraction>

</dg:chunk>
```

`InvoiceTotal` has no `dg:origin` — this invoice doesn't print a total — and carries `dg:method="calculated"` because its value is calculated: `(3 × $49.95) + (1 × $200.00) = $349.85`. `dg:inputs` lists all four elements it used, and the derivation rule is the field's `## Prompt:` in the schema (§12), so a verifier can redo the calculation.

Equivalent JSON-LD:

```
{
  "@context": {
    "dg":      "http://dgml.io/ns/dg#",
    "docset":  "http://www.dgml.io/acme/invoices#",
    "xsd":     "http://www.w3.org/2001/XMLSchema#"
  },
  "@type": "dg:chunk",
  "dg:extraction": {
    "@type": "docset:Invoice",
    "docset:VendorName": {
      "@value": "MagicSoft, Inc.",
      "dg:origin": "1 220 150 680 200"
    },
    "docset:InvoiceDate": {
      "@type": "xsd:date",
```

```
      "@value": "2025-03-01",
      "dg:origin": "1 900 150 1300 200"
    },
    "docset:LineItems": {
      "@type": "docset:LineItems",
      "docset:LineItem": [
        {
          "@id": "#ln1",
          "docset:ProductName": { "@value": "Widget Pro",  "@id": "#ln1-name", "dg:origin": "2 2
          "docset:Quantity":    { "@type": "xsd:integer", "@value": "3",       "@id": "#ln1-qty"
          "docset:UnitPrice":   { "@type": "xsd:decimal", "@value": "49.95",   "@id": "#ln1-pric
        },
        {
          "@id": "#ln2",
          "docset:ProductName": { "@value": "Support Plan", "@id": "#ln2-name", "dg:origin": "2
          "docset:Quantity":    { "@type": "xsd:integer", "@value": "1",        "@id": "#ln2-qty
          "docset:UnitPrice":   { "@type": "xsd:decimal", "@value": "200.00",   "@id": "#ln2-pri
        }
      ]
    },
    "docset:InvoiceTotal": {
      "@type": "xsd:decimal",
      "@value": "349.85",
      "dg:method": "calculated",
      "dg:inputs": [
        { "@id": "#ln1-qty" }, { "@id": "#ln1-price" },
        { "@id": "#ln2-qty" }, { "@id": "#ln2-price" }
      ]
    }
  }
}
```

Part II — DGMLX

15. What DGMLX Is

DGMLX (`.dgmlx`) is a ZIP archive that packages a DGML document together with everything needed to verify and use it:

- The source document — PDF, DOCX, PPTX, TIFF, HTML, or any format that can be rendered to page images

- Page images rendered from the source
- The DGML XML
- A RELAX NG schema defining the docset vocabulary
- OPC package relationships
- An attestation document with provenance metadata

DGMLX uses OPC (Open Packaging Conventions, ECMA-376 Part 2) as its container convention.

16. Package Structure

```
[Content_Types].xml               ← OPC content-type registry
_rels/.rels                       ← OPC package relationships
META-INF/dgml-attestation.xml    ← Attestation (Merkle root, DPI, renderer, IDs)
schema.rnc                        ← RELAX NG schema for the docset vocabulary
<Document Title>.dgml.xml         ← Semantic XML (the DGML document)
source/<Document Title>.<ext>     ← Original source document
page_images/page_1.png            ← Rendered page image, page 1
page_images/page_2.png
...
page_images/page_N.png
```

Rules:

- The document title (filename stem) is the same across `<title>.dgml.xml` and `source/<title>.<ext>`.
- Page image files are named `page_N.png` where N is a 1-based integer with no leading zeros.
- `META-INF/` is reserved. Only `dgml-attestation.xml` should be placed there.
- `_rels/` is the OPC reserved directory for relationship files.
- No `page_text/` directory is present — token data is an intermediate artifact, not shipped.

17. Container — `[Content_Types].xml`

Follows OPC §10.1. Declares the MIME type for every part in the package.

```
<?xml version="1.0" encoding="UTF-8"?>
<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">
  <Default Extension="rnc" ContentType="text/plain"/>
  <Default Extension="png" ContentType="image/png"/>
```

```
  <Default Extension="xml" ContentType="application/xml"/>
  <Override PartName="/_rels/.rels"
            ContentType="application/vnd.openxmlformats-package.relationships+xml"/>
  <Override PartName="/META-INF/dgml-attestation.xml" ContentType="application/xml"/>
  <!-- One Override per source file and page image with exact PartName -->
</Types>
```

Source file content types:

Source format	Content type
PDF	`application/pdf`
DOCX	`application/vnd.openxmlformats-officedocument.wordprocessingml.document`
PPTX	`application/vnd.openxmlformats-officedocument.presentationml.presentation`
TIFF	`image/tiff`

18. Package Relationships — `_rels/.rels`

Follows OPC §9. Typed relationships from the package root to every significant part.

Type URI	Target
`http://dgml.io/relationships/attestation`	`META-INF/dgml-attestation.xml`
`http://dgml.io/relationships/source`	Source document
`http://dgml.io/relationships/semantic-xml`	DGML XML file
`http://dgml.io/relationships/schema`	`schema.rnc`
`http://dgml.io/relationships/page-image`	Each page PNG (one per page)

```
<?xml version="1.0" encoding="UTF-8"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId1" Type="http://dgml.io/relationships/attestation"
                Target="META-INF/dgml-attestation.xml"/>
  <Relationship Id="rId2" Type="http://dgml.io/relationships/source"
                Target="source/Acme Corp MSA 2024.pdf"/>
  <Relationship Id="rId3" Type="http://dgml.io/relationships/semantic-xml"
```

```
                Target="Acme Corp MSA 2024.dgml.xml"/>
  <Relationship Id="rId4" Type="http://dgml.io/relationships/schema"
                Target="schema.rnc"/>
  <Relationship Id="IMG1" Type="http://dgml.io/relationships/page-image"
                Target="page_images/page_1.png" Order="1"/>
  <Relationship Id="IMG2" Type="http://dgml.io/relationships/page-image"
                Target="page_images/page_2.png" Order="2"/>
</Relationships>
```

Page-image `Id` values are `IMG1`, `IMG2`, … matching the page number. `Order` is a DGML extension attribute. All `Target` paths are relative to the package root.

19. Attestation — `META-INF/dgml-attestation.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<dgml-attestation xmlns="http://dgml.io/ns/attestation"
                  version="1"
                  page-image-dpi="300"
                  page-image-renderer="ghostscript"
                  file-id="5kqt9r5fowno"
                  docset-id="o8vr8rs488vg">
  <merkle-root>62f7a978b4b1b3f1eaf400514070e827873bed8b4fb4279bb4d6e7288f79df8e</merkle-root>
</dgml-attestation>
```

For a package that is part of a provenance chain (Section 11), the attestation also includes one `<prior-merkle-root>` per predecessor:

```
<dgml-attestation xmlns="http://dgml.io/ns/attestation"
                  version="1" page-image-dpi="150"
                  page-image-renderer="ghostscript"
                  file-id="amend-001" docset-id="o8vr8rs488vg">
  <merkle-root>a3f9...</merkle-root>
  <prior-merkle-root file-id="5kqt9r5fowno">62f7a978...</prior-merkle-root>
</dgml-attestation>
```

Fields

Attribute / Element	Meaning
`version`	Attestation format version (currently `"1"`)
`file-id`	Unique identifier for this document file (permanent)
`docset-id`	Identifier of the docset this file belongs to (permanent)
`page-image-dpi`	DPI at which the source was rendered. All `dg:origin` coordinates reference this. **Default: 150.**
`page-image-renderer`	Tool used to render pages (`"ghostscript"`, `"libreoffice"`, `"pillow"`, ...)
`<merkle-root>`	SHA-256 Merkle root over the canonical content of all page images
`<prior-merkle-root>`	Merkle root of a predecessor package (for provenance chains); `file-id` attribute identifies the predecessor

20. Page Images — `page_images/page_N.png`

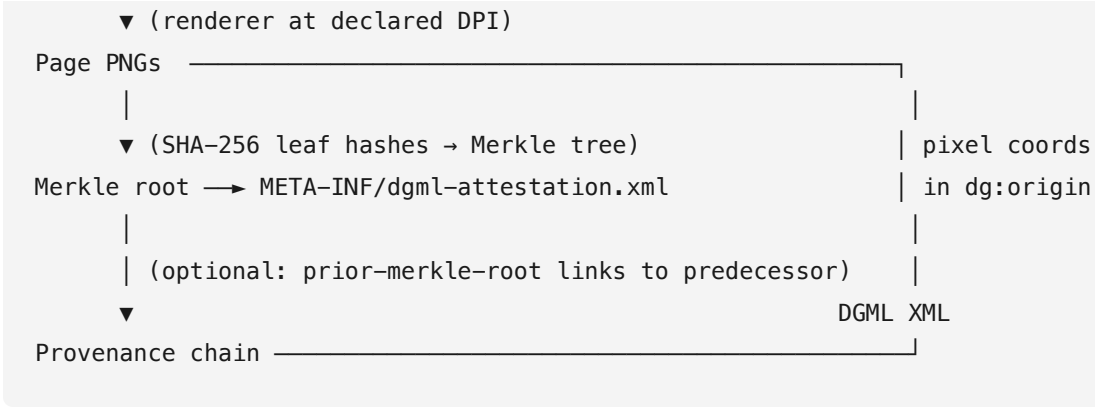
Lossless PNG renders of each page. Named `page_N.png`, 1-based, no leading zeros. Rendered at the DPI declared in attestation using the declared renderer (Ghostscript for PDF, LibreOffice for DOCX/PPTX, Pillow for raster sources, etc.).

Recommended DPI values:

DPI	Letter page pixel size	Typical use
150	1275 × 1650	Default — sufficient for LLM grounding
200	1700 × 2200	Higher fidelity for dense documents
300	2550 × 3300	High quality, standard OCR DPI

20. Verification

```
Source document
      |
```



1. **Integrity** — re-render at declared DPI with declared renderer, recompute Merkle root, compare to attestation.
2. **Origin accuracy** — crop `dg:origin` rectangle from `page_images/page_N.png`, confirm it shows the element's text.
3. **Schema conformance** — validate `.dgml.xml` against `schema.rnc` with any RELAX NG validator (e.g., Jing).
4. **Provenance** — for each value with `dg:inputs`, confirm every target exists, reapply the field's `## Prompt:` rule, and recompute `calculated` values (§13).
5. **Chain integrity** — if `<prior-merkle-root>` is present, verify the predecessor package and confirm its `<merkle-root>` matches.

Appendix A — JSON-LD

A.1 Semantic Links

`dg:itemprop` / `dg:href` pairs form subject–predicate–object triples that map directly to JSON-LD. The subject is the element carrying the attributes (identified by its `xml:id`); the predicate is the `dg:itemprop` value; the object is the element identified by `dg:href`.

Input XML:

```
<docset:VendorName xml:id="vendor-co"
    dg:origin="1 220 150 680 200">MagicSoft, Inc.</docset:VendorName>

<docset:SignatoryName
    xml:id="sig-1"
    dg:itemprop="signatoryOf"
    dg:href="#vendor-co"
    dg:origin="11 460 2430 1200 2475">Jane Smith</docset:SignatoryName>
```

JSON-LD triple produced by the document instance export (A.2):

```
{
  "@id": "#sig-1",
  "@type": "docset:SignatoryName",
  "signatoryOf": { "@id": "#vendor-co" }
}
```

The full translation rules are defined in A.2.

A.2 Document Instance Export (XAST)

The DGML XML file is the system of record. JSON-LD representations are exports derived directly from the XML, acting as delivery vehicles optimized for specific consumers and transport layers. This specification defines one export: the XAST-convention lossless export. The community may define additional exports derived directly from XML for specific consumers; those are out of scope for this specification.

The XAST export follows the conventions of XAST (XML Abstract Syntax Tree), a DOM-style tree representation of XML in JSON popularized by the unified/xast ecosystem (https://github.com/syntax-tree/xast). Every node has a `type`, optional `children` array, and optional `attributes` — making the structure predictable and lossless.

DGML extends this convention with JSON-LD identity and linking: `@context` for namespace mappings, `@type` for semantic tag names, `@id` for stable node identity, and named link properties derived from `dg:itemprop` / `dg:href` stubs.

Output shape:

```
{
  "@context": {
    "xast": "http://dgml.io/ns/xast#",
    "children":   { "@id": "xast:children",   "@container": "@list"  },
    "attributes": { "@id": "xast:attributes", "@container": "@index" },
    "nodeType":   { "@id": "xast:nodeType" },
    "value":      { "@id": "xast:value" },
    "...namespace prefix mappings..."
  },
  "@graph": [ "<root node>" ]
}
```

The `xast:` terms are always present, defined inline — no external fetch required. `@container: "@list"` on `children` preserves document order for JSON-LD processors. `@container: "@index"` on `attributes` makes each attribute entry individually visible to JSON-LD processors.

Node shape — every XML element becomes a JSON object with:

- `"nodeType": "xast:element"` — always present, fixed string; distinguishes element nodes from text nodes for JSON-LD processors.
- `"@type": "<curie>"` — the element tag as a CURIE (e.g. `"docset:LiabilityCap"`, `"dg:chunk"`)
- `"@id": "..."` — only present if the element has an `id` or `xml:id` attribute. Plain local ids (no colon, slash, or leading `#`) are prefixed with `#`; URNs and full URIs are left unchanged.
- `"attributes": { ... }` — all XML attributes except `id` / `xml:id`, `dg:itemprop`, `dg:href`, and `dg:inputs`, keyed by their CURIE name, values as strings. `dg:origin`, `dg:method`, `dg:structure`, and `dg:style` are included here as regular attributes. Omit key entirely if no attributes remain.
- `"children": [ ... ]` — ordered array of child nodes (element nodes and text nodes), always present, always in document order, may be empty.
- Named link property — if the element itself carries `dg:itemprop` and `dg:href`, those become a named property on this node: `{ "<dg:itemprop_value>": {"@id": "<dg:href_value>"} }`. `dg:itemprop` and `dg:href` are never included in `"attributes"`. The element still appears in `children` normally — nothing is consumed or hidden.
- Inputs property — if the element carries `dg:inputs`, it becomes `"dg:inputs": [ {"@id": "<ref>"}, ... ]`, one object per reference in order (§13). `dg:inputs` is never included in `"attributes"`.

Text node shape:

```
{"nodeType": "xast:text", "value": "..."}
```

Context rules:

- Each `xmlns:prefix="uri"` declaration → `"@context"` entry `"prefix": "uri"`
- Default namespace `xmlns="uri"` → `"@vocab": "uri"` in `@context`
- No named prefixes → no `@vocab`

Itemprop/href rules: If an element carries both `dg:itemprop` and `dg:href` attributes, those attributes produce a named link property on that element — `{ dg:itemprop_value: {"@id": dg:href_value} }`. The element appears in `children` in the normal position; `dg:itemprop` and `dg:href` are excluded from `"attributes"`. Nothing is consumed or re-parented.

Example:

Input XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<dg:chunk xmlns:dg="http://dgml.io/ns/dg#"
          xmlns:docset="http://www.dgml.io/acme-corp/master-services-agreements#"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <docset:IndemnificationClause dg:origin="3 180 400 2360 1200">
    <dg:chunk dg:structure="header" dg:style="font-size: 1.5em; text-transform: uppercase"
              dg:origin="3 180 400 2360 450">
      <dg:chunk dg:structure="lim">3.1</dg:chunk>
      Indemnification
    </dg:chunk>
    <docset:LiabilityCap xsi:type="decimal" dg:value="500000"
        dg:origin="3 180 500 2360 540">$500,000</docset:LiabilityCap>
    <docset:EffectiveDate xsi:type="date" dg:value="2024-01-01"
        dg:origin="3 180 550 2360 590">January 1, 2024</docset:EffectiveDate>
    <docset:VendorRef dg:itemprop="docset:paidBy" dg:href="#vendor-co"/>
  </docset:IndemnificationClause>
  <docset:VendorName xml:id="vendor-co"
      dg:origin="1 220 150 680 200">MagicSoft, Inc.</docset:VendorName>
</dg:chunk>
```

Expected JSON-LD output:

```
{
  "@context": {
    "xast": "http://dgml.io/ns/xast#",
    "children":   { "@id": "xast:children",   "@container": "@list"  },
    "attributes": { "@id": "xast:attributes", "@container": "@index" },
    "nodeType":   { "@id": "xast:nodeType" },
    "value":      { "@id": "xast:value" },
    "dg":     "http://dgml.io/ns/dg#",
    "docset": "http://www.dgml.io/acme-corp/master-services-agreements#",
    "xsi":    "http://www.w3.org/2001/XMLSchema-instance"
  },
  "@graph": [
    {
      "nodeType": "xast:element",
      "@type": "dg:chunk",
      "children": [
        {
          "nodeType": "xast:element",
          "@type": "docset:IndemnificationClause",
```

```
          "attributes": {
            "dg:origin": "3 180 400 2360 1200"
          },
          "children": [
            {
              "nodeType": "xast:element",
              "@type": "dg:chunk",
              "attributes": {
                "dg:structure": "header",
                "dg:style": "font-size: 1.5em; text-transform: uppercase",
                "dg:origin": "3 180 400 2360 450"
              },
              "children": [
                {
                  "nodeType": "xast:element",
                  "@type": "dg:chunk",
                  "attributes": {"dg:structure": "lim"},
                  "children": [
                    {"nodeType": "xast:text", "value": "3.1"}
                  ]
                },
                {"nodeType": "xast:text", "value": "Indemnification"}
              ]
            },
            {
              "nodeType": "xast:element",
              "@type": "docset:LiabilityCap",
              "attributes": {
                "xsi:type": "decimal",
                "dg:value": "500000",
                "dg:origin": "3 180 500 2360 540"
              },
              "children": [
                {"nodeType": "xast:text", "value": "$500,000"}
              ]
            },
            {
              "nodeType": "xast:element",
              "@type": "docset:EffectiveDate",
              "attributes": {
                "xsi:type": "date",
                "dg:value": "2024-01-01",
                "dg:origin": "3 180 550 2360 590"
              },
              "children": [
                {"nodeType": "xast:text", "value": "January 1, 2024"}
              ]
```

```
            },
            {
              "nodeType": "xast:element",
              "@type": "docset:VendorRef",
              "docset:paidBy": {"@id": "#vendor-co"},
              "children": []
            }
          ]
        },
        {
          "nodeType": "xast:element",
          "@type": "docset:VendorName",
          "@id": "#vendor-co",
          "attributes": {
            "dg:origin": "1 220 150 680 200"
          },
          "children": [
            {"nodeType": "xast:text", "value": "MagicSoft, Inc."}
          ]
        }
      ]
    }
  ]
}
```

A.3 Docset Schema: JSON-LD Context

A docset schema (written in RELAX NG Compact syntax, `schema.rnc`) translates mechanically to a JSON-LD document that declares the vocabulary used by that docset. The result contains a fixed `@context` block and a `@graph` of tag and group definitions.

Concept mapping

RNC construct	JSON-LD result
`namespace docset = "…"`	`"docset": "…"` entry in `@context`
Element definition	Node with `"@type": "docset:Tag"`
Named group (alternation of element refs)	Node with `"@type": "docset:TagGroup"` and a `"members"` array
`## description line` doc comment	`"description"` property on the tag node

RNC construct	JSON-LD result
`## Example: …` doc comment line	`"example"` property on the tag node
`## Prompt: …` doc comment line	`"prompt"` property on the tag node
Element content model (child element refs)	`"children"` array of tag `@id` references on the tag node

Fixed @context shape

Every docset schema produces a `@context` with the following fixed keys in addition to the docset namespace prefix:

```
{
  "@context": {
    "docset": "<namespace URI from schema>",
    "xsd": "http://www.w3.org/2001/XMLSchema#",
    "Tag":          "docset:Tag",
    "TagGroup":     "docset:TagGroup",
    "members":      { "@id": "docset:members", "@type": "@id", "@container": "@set" },
    "description":  "docset:description",
    "example":      "docset:example",
    "prompt":       "docset:prompt",
    "children":     { "@id": "docset:children", "@type": "@id", "@container": "@list" }
  },
  "@graph": [ "..." ]
}
```

`Tag`, `TagGroup`, `members`, `description`, `example`, `prompt`, and `children` are always present. They expand against the docset namespace, so every docset gets its own copy of these terms in its namespace — they are not shared across docsets.

Example

The following RNC schema defines the vocabulary for the same ACME Corp Master Services Agreements docset used in the A.2 example. The tags here correspond directly to the `docset:` elements in that XML instance.

Input RNC schema:

```
namespace docset = "http://www.dgml.io/acme-corp/master-services-agreements#"

ClauseItemTag = LiabilityCap | EffectiveDate | VendorName | VendorRef

## Indemnification clause section
IndemnificationClause =
  element docset:IndemnificationClause {
    (text | ClauseItemTag)*
  }

## Agreed maximum liability exposure under the contract
## Example: $500,000
LiabilityCap =
  element docset:LiabilityCap {
    (text | ClauseItemTag)*
  }

## Effective date of the contract
## Example: January 1, 2024
EffectiveDate =
  element docset:EffectiveDate {
    (text | ClauseItemTag)*
  }

## Legal name of the vendor party
## Example: MagicSoft, Inc.
VendorName =
  element docset:VendorName {
    (text | ClauseItemTag)*
  }

## Reference to a vendor entity
VendorRef =
  element docset:VendorRef {
    (text | ClauseItemTag)*
  }
```

Expected JSON-LD output:

```
{
  "@context": {
    "docset": "http://www.dgml.io/acme-corp/master-services-agreements#",
    "xsd": "http://www.w3.org/2001/XMLSchema#",
    "Tag":          "docset:Tag",
    "TagGroup":     "docset:TagGroup",
```

```
    "members":      { "@id": "docset:members", "@type": "@id", "@container": "@set" },
    "description":  "docset:description",
    "example":      "docset:example"
  },
  "@graph": [
    {
      "@id": "docset:ClauseItemTag",
      "@type": "docset:TagGroup",
      "members": [
        "docset:LiabilityCap",
        "docset:EffectiveDate",
        "docset:VendorName",
        "docset:VendorRef"
      ]
    },
    {
      "@id": "docset:IndemnificationClause",
      "@type": "docset:Tag",
      "description": "Indemnification clause section"
    },
    {
      "@id": "docset:LiabilityCap",
      "@type": "docset:Tag",
      "description": "Agreed maximum liability exposure under the contract",
      "example": "$500,000"
    },
    {
      "@id": "docset:EffectiveDate",
      "@type": "docset:Tag",
      "description": "Effective date of the contract",
      "example": "January 1, 2024"
    },
    {
      "@id": "docset:VendorName",
      "@type": "docset:Tag",
      "description": "Legal name of the vendor party",
      "example": "MagicSoft, Inc."
    },
    {
      "@id": "docset:VendorRef",
      "@type": "docset:Tag",
      "description": "Reference to a vendor entity"
    }
  ]
}
```

The `docset:` namespace URI here is identical to the one declared on the XML instance in A.2, so `@type: "docset:LiabilityCap"` in the XAST export expands to the same IRI as `@id: "docset:LiabilityCap"` in the schema — making the two JSON-LD documents directly joinable by a standard JSON-LD processor.

`Tag` nodes identify individual schema fields — e.g., a liability cap amount, an effective date, or a vendor name. `TagGroup` nodes define named sets of tags that appear together in a given structural role, such as all tags valid inside a clause.

Appendix B — Minimal Complete Example

Two-page MSA, organization `acme-corp`, docset `master-services-agreements`:

```
[Content_Types].xml
_rels/.rels
META-INF/dgml-attestation.xml
schema.rnc
Acme Corp MSA 2024.dgml.xml
source/Acme Corp MSA 2024.pdf
page_images/page_1.png
page_images/page_2.png
```

`META-INF/dgml-attestation.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<dgml-attestation xmlns="http://dgml.io/ns/attestation"
                  version="1" page-image-dpi="150"
                  page-image-renderer="ghostscript"
                  file-id="abc123" docset-id="xyz456">
  <merkle-root>deadbeef...</merkle-root>
</dgml-attestation>
```

`Acme Corp MSA 2024.dgml.xml` (excerpt)

```
<?xml version='1.0' encoding='utf-8'?>
<dg:chunk xmlns:dg="http://dgml.io/ns/dg#"
          xmlns:docset="http://www.dgml.io/acme-corp/master-services-agreements#"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
```

```
  <docset:VendorName xml:id="vendor"
      dg:origin="1 220 150 680 200">MagicSoft, Inc.</docset:VendorName>
  <docset:IndemnificationClause dg:origin="2 460 300 1800 800">
    <docset:LiabilityCap xsi:type="decimal" dg:value="500000"
        dg:origin="2 460 310 1800 355">$500,000</docset:LiabilityCap>
    <docset:EffectiveDate xsi:type="date" dg:value="2024-01-01"
        dg:origin="2 998 710 1466 755">January 1, 2024</docset:EffectiveDate>
  </docset:IndemnificationClause>
  <docset:PaymentTerms>
    <docset:InvoiceCycle dg:origin="4 460 139 1800 184">Net 30</docset:InvoiceCycle>
    <docset:SignatoryName
        dg:itemprop="signatoryOf" dg:href="#vendor"
        dg:origin="11 460 2430 1200 2475">Jane Smith</docset:SignatoryName>
  </docset:PaymentTerms>
</dg:chunk>
```

Appendix C — Changes in 0.71

0.71 changes how DGML records values a page prints only in part, values it doesn't print at all, and values in born-digital documents. It is a **breaking change for readers**: a 0.7 reader would take a value printed in part for a fully printed one.

0.7	0.71
`dg:origin="computed"` marks a derived value	`dg:origin` holds boxes only; a value the page doesn't print has none, and `dg:method` says how it was produced (§7, §13)
`dg:itemprop="computedFrom"` + `dg:href` name a derived value's sources	`dg:inputs` names the elements any value used; `dg:itemprop` / `dg:href` are semantic links only (§8, §13)
A value printed in part needs a raw companion element plus a computed one	One element: boxes for the printed part, `dg:inputs` for what completed it
No answer for born-digital values	They omit `dg:origin` (§7, §13)
Inputs that aren't schema fields can't be recorded	Anonymous `dg:chunk` elements carry them (§13 rule 7)

Reading 0.7 files. When an element has `dg:origin="computed"`, ignore that origin and read its `dg:href` targets as `dg:inputs`. Every other 0.7 element reads the same in 0.71.

Migration, readers first:

1. Readers apply the rule above, split reference lists on semicolons or whitespace, and treat a missing `dg:method` as described in §13 rule 4.
2. Producers stop writing `"computed"`, emit `dg:inputs`, fold each raw companion into its value element, write `dg:method` on every value the page doesn't print, and emit a `dg:chunk` for each input that isn't a schema field.
3. Schemas that list allowed attributes add `dg:inputs` and `dg:method`; schemas that list allowed elements also allow `dg:chunk` next to the fields.
4. JSON-LD exports `dg:inputs` as an array of `{"@id"}` objects; `dg:method` stays a plain attribute (Appendix A.2).

Rewriting an element changes its hash and the file's Merkle root, so anchored files (§10) must be re-anchored, or left in the 0.7 form.

End of DGML Format Specification